Gitfolio

GitHub Portfolio Analyzer

Transform a GitHub profile into a job-ready, recruiter-approved portfolio assessment in under 60 seconds — powered by AI.

Technical Documentation

Version 1.0.0 · June 2026

React + Node.js (Express) + Python (FastAPI)

Table of Contents

1. Project Overview

Gitfolio (internally "GitHub Portfolio Analyzer") is an AI-powered tool that evaluates GitHub profiles and turns them into recruiter-style feedback. For students it answers *"How strong does my GitHub look, and how do I improve it?"*; for recruiters it answers *"Which of these candidates actually match the role?"*

The Problem

For early-career developers, GitHub is the de-facto portfolio — but most profiles fail to communicate real skill, impact, or consistency. Recruiters struggle with missing READMEs, weak repo structure, inconsistent commit history, and no clear signal of technical depth. Strong developers get overlooked.

The Solution

Gitfolio fetches live GitHub data, computes a multi-dimensional portfolio score, and runs an LLM to produce structured, actionable feedback — strengths, red flags, and an improvement roadmap.

Key Features

Feature	Audienc	What it does

e

GitHub Profile Analysis	Student	Scores a profile out of 100 across 5 dimensions and generates AI feedback (strengths, red flags, action plan, top repos).
Specialization Fit Checker	Student	Scores how well a profile fits a target role (e.g. Backend, ML, DevOps) against a job description, with a learning path and radar chart.
Candidate Shortlist	Recruiter	Upload an Excel roster of candidates + a job description; returns an AI-ranked shortlist of the top N.
Resume vs GitHub Match	Recruiter	Upload a resume (PDF/DOCX/TXT) + a GitHub URL; checks whether the resume claims are backed by real GitHub evidence.
GitHub Profile Compare	Recruiter	Compares two GitHub profiles head-to-head with scored breakdowns.

> **Architecture in one line:** a React UI talks to a Node/Express API, which fetches and compresses GitHub data and then forwards it to a Python/FastAPI service that runs the LLM. The Node layer never calls the LLM directly; the Python layer never calls GitHub directly.

2. Tech Stack

Frontend — `client/`

Tool	Version	Purpose
React	19	UI library
Vite	7	Dev server & build tool
Tailwind CSS	4	Utility-first styling (via `@tailwindcss/vite`)
Framer Motion	12	Animations & page transitions
Recharts	3	Radar / score charts

React Icons	5	Icon set (Feather icons)
Axios	1	HTTP client

Backend API — `server/`

Tool	Version	Purpose
Node.js + Express	5	HTTP API framework
Helmet	8	Security headers
express-rate-limit	8	Rate limiting (100 req / 15 min)
compression	1	Gzip responses
morgan	1	HTTP request logging
cors	2	Cross-origin policy
multer	2	In-memory file uploads (resume / Excel)
unpdf + mammoth	1	PDF and DOCX text extraction
exceljs	4	Excel roster parsing
axios	1	Calls GitHub REST API & the ML service
dotenv	17	Environment variables

AI Service — `ml-service/`

Tool	Version	Purpose
Python	3.8+	Language
FastAPI	0.115	Async web framework

Uvicorn	0.32	ASGI server
OpenAI SDK	1.57	Client, pointed at NVIDIA's OpenAI-compatible endpoint
Pydantic	2.10	Request/response validation & data contracts
python-dotenv	1.0	Environment variables

> **LLM provider:** Despite the OpenAI SDK, the service calls **NVIDIA's hosted inference API** (`https://integrate.api.nvidia.com/v1`) using the model `openai/gpt-oss-120b` by default. Authentication uses `NVIDIA_API_KEY` — *not* an OpenAI key. (An older README mentions OpenAI GPT-4o-mini; the live code uses NVIDIA.)

Tooling

- **concurrently** — runs frontend + backend together from the repo root.
- **nodemon** — server auto-reload in development.
- **ESLint** — frontend linting.

3. Folder Structure

```
Github-Project/
├── package.json              # Root scripts: run all 3 services via
concurrently
│
├── client/                   # ── React + Vite frontend ──
│   ├── index.html
│   ├── vite.config.js
│   ├── eslint.config.js
│   └── src/
│       ├── main.jsx          # React entry point
│       ├── App.jsx           # Wraps the app in an ErrorBoundary
│       ├── routes.js         # Centralized route path constants
│       ├── index.css         # Tailwind layer + globals
│       ├── pages/            # Page-level components (one per screen)
│       │   ├── Dashboard.jsx          # Custom history-based router/shell
│       │   ├── HomePage.jsx
│       │   ├── student/               # Student-facing pages
│       │   └── hr/                    # Recruiter-facing pages
│       ├── components/       # Shared UI (buttons, loaders, cards)
```

```
│       ├── features/          # Self-contained feature modules:
│       │   ├── specialization/        #   each has a hook + components/
│       │   ├── resumeMatch/
│       │   ├── githubCompare/
│       │   └── shortlist/
│       └── services/
│           └── api.js         # Axios clients + one function per endpoint
│
├── server/                    # ── Node.js + Express API ──
│   ├── server.js              # Boot + graceful shutdown
│   └── src/
│       ├── app.js             # Express app: middleware, CORS, routes wiring
│       ├── routes/            # One router per feature
│       ├── controllers/       # Request handlers (orchestration)
│       ├── services/          # Business logic (GitHub fetch, scoring, analysis)
│       ├── middleware/        # Validation + error handling
│       └── utils/             # Helpers (parsers, cache, username extractor)
│
└── ml-service/                # ── Python + FastAPI AI engine ──
    ├── main.py                # FastAPI app + all endpoints
    ├── analyzer.py            # Base GitHub-analysis feedback generation
    ├── specialization/        # Per-feature package: schema, service, prompts
    ├── resume_match/          #   "
    ├── github_compare/        #   "
    └── shortlist/             #   "
        ├── schema.py          # Pydantic request/response models
        ├── service.py         # Orchestrates the LLM call
        └── prompt_builder.py # Builds the system/user prompt
```

Why this structure

The server uses a conventional *routes → controllers → services* layering. The client and ML service are organized **by feature**: each feature (specialization, resume match, etc.) keeps its UI, hook, schema, and prompt logic together, so adding a new feature means adding one folder per layer rather than editing shared files.

4. Installation & Setup

Prerequisites

- **Node.js 18+** and npm
- **Python 3.8+** and pip

- **NVIDIA API key** — required for all AI feedback (build.nvidia.com)
- **GitHub Personal Access Token** — optional, but strongly recommended to avoid GitHub's 60 req/hr unauthenticated rate limit

Step 1 — Clone & install root tooling

```
git clone https://github.com/ArmanSunasara/Gitfolio.git
cd Gitfolio
npm install        # installs "concurrently" used by the root start script
```

Step 2 — Backend (server)

```
cd server
npm install
```

Create `server/.env` (see Environment Variables):

```
PORT=5000
NODE_ENV=development
GITHUB_TOKEN=ghp_your_token_here
ML_SERVICE_URL=http://localhost:8000
FRONTEND_URL=http://localhost:5173
```

Step 3 — Frontend (client)

```
cd ../client
npm install
```

Create `client/.env`:

```
VITE_API_URL=http://localhost:5000
```

Step 4 — AI service (ml-service)

```
cd ../ml-service
python -m venv .venv
# Windows:  .venv\Scripts\activate
# macOS/Linux:  source .venv/bin/activate
```

```
pip install -r requirements.txt
```

Create `ml-service/.env`:

```
NVIDIA_API_KEY=nvapi-your_key_here
NVIDIA_MODEL=openai/gpt-oss-120b
CORS_ORIGINS=http://localhost:5000,http://localhost:5173
```

Step 5 — Run everything

The ML service is started separately (Python); the frontend and backend can be started together from the repo root:

```
# Terminal 1 — AI service
cd ml-service
uvicorn main:app --reload --port 8000

# Terminal 2 — frontend + backend together (from repo root)
npm start
#   └─ runs:  client (vite)  +  server (node)
```

Or start each individually:

```
npm run frontend     # client on http://localhost:5173
npm run backend      # server on http://localhost:5000
npm run ml-service   # ml-service on http://localhost:8000
```

> Open **http://localhost:5173** in your browser. The app works without the ML service running, but AI feedback will be empty/unavailable.

5. Environment Variables

Server — `server/.env`

Variable	Required	Example	Description
`PORT`	No	`5000`	Port the Express API listens on (defaults to 5000).

`NODE_ENV`	No	`development`	Switches logging verbosity and error detail. Use `production` in prod.
`GITHUB_TOKEN`	Recommended	`ghp_xxx`	GitHub PAT for higher API rate limits. Without it, GitHub allows only 60 requests/hour.
`ML_SERVICE_URL`	Yes (for AI)	`http://localhost:8000`	Base URL of the Python ML service. If unset, the API still works but returns no AI feedback.
`FRONTEND_URL`	Recommended	`http://localhost:5173`	Comma-separated allowed CORS origins.

Client — `client/.env`

Variable	Required	Example	Description
`VITE_API_URL`	No	`http://localhost:5000`	Base URL of the Node API. Defaults to `http://localhost:5000`. Must be prefixed `VITE_` to be exposed to the browser.

ML Service — `ml-service/.env`

Variable	Required	Example	Description
`NVIDIA_API_KEY`	Yes	`nvapi-xxx`	API key for NVIDIA's hosted inference. The service raises an error on first LLM call if missing.
`NVIDIA_MODEL`	No	`openai/gpt-oss-120b`	Model name. Defaults to `openai/gpt-oss-120b`.
`CORS_ORIGINS`	No	`http://localhost:5000,...`	Comma-separated allowed origins for the FastAPI CORS middleware.

> **Never commit real secrets.** All `.env` files are gitignored. The values above are placeholders.

6. Routing & API Endpoints

6.1 Frontend routes (client-side)

The client uses a lightweight custom router (`Dashboard.jsx` + `routes.js`) built on the History API — there is no React Router. Route constants live in one place:

Path	Screen
`/`	Home (Student vs Recruiter split)
`/students`	Student tools menu
`/students/ats-score`	GitHub Profile Analysis (ATS score)
`/students/specialization-fit`	Specialization Fit Checker
`/recruiters`	Recruiter tools menu
`/recruiters/candidate-shortlist`	Candidate Shortlist
`/recruiters/resume-github-match`	Resume vs GitHub Match
`/recruiters/github-profile-compare`	GitHub Profile Compare

6.2 Backend API (Node — base `http://localhost:5000`)

All endpoints return `{ "success": true, "data": ... }` on success or `{ "success": false, "error": "..." }` on failure. The whole `/api` surface is rate-limited to 100 requests / 15 minutes.

Method	Endpoint	Body	Purpose
GET	`/api/health`	—	Health/uptime check.
POST	`/api/github/analyze`	JSON `{ url }`	Analyze one profile; returns score + repo analysis + AI feedback.

GET	`/api/specialization/roles`	—	List the supported target roles for the dropdown.
POST	`/api/specialization/analyze`	JSON `{ url, roleId, jobDescription }`	Score role fit + learning path.
POST	`/api/shortlist/analyze`	multipart `excel_file`, `jobDescription`, `shortlistCount`	Rank a roster and return top N.
POST	`/api/resume-match/analyze`	multipart `resume`, `url`	Match a resume against GitHub evidence.
POST	`/api/github-compare/analyze`	JSON `{ profileA, profileB }`	Compare two profiles head-to-head.

Validation rules (enforced by middleware)

- **GitHub URL/username**: required, ≤ 256 chars; accepts a full URL or a bare handle.
- **jobDescription**: 20–8000 characters.
- **roleId**: must be one of the 19 known roles (e.g. `backend_engineer`, `ml_engineer`, `devops_engineer`).
- **shortlistCount**: integer 1–100.
- **resume**: `.pdf`, `.docx`, `.txt`, or `.md`, ≤ 5 MB.
- **excel_file**: `.xlsx`, `.xls`, or `.xlsm`, ≤ 5 MB.
- **compare**: profileA and profileB must be different handles.

Example — analyze a profile

```
curl -X POST http://localhost:5000/api/github/analyze \
  -H "Content-Type: application/json" \
  -d '{"url": "https://github.com/torvalds"}'
```

Response (abbreviated):

```
{
  "success": true,
```

```
  "username": "torvalds",
  "avatarUrl": "https://avatars.githubusercontent.com/u/1024025",
  "publicRepos": 8,
  "followers": 230000,
  "score": 82,
  "repoAnalysis": [
    { "name": "linux", "stars": 180000, "hasReadme": true,
      "commitCount": 100, "languages": ["C", "Assembly"] }
  ],
  "aiFeedback": {
    "strengths": ["Massive community impact", "..."],
    "redFlags": ["Few documented side projects"],
    "suggestions": ["Add per-repo READMEs", "..."]
  }
}
```


6.3 ML Service API (Python — base `http://localhost:8000`)

These are **internal** endpoints called by the Node server, not by the browser. Each returns `{ "success": true, "data": ... }`.

Method	Endpoint	Purpose
GET	`/health`	Service health check.
POST	`/analyze`	Generate base GitHub feedback from score + repo analysis.
GET	`/specialization/roles`	Authoritative list of roles.
POST	`/specialization/analyze`	Role-fit report.
POST	`/resume-match/analyze`	Resume vs GitHub report.
POST	`/github-compare/analyze`	Two-profile comparison.
POST	`/shortlist/analyze`	Rank a batch of candidates.

7. Core Modules & How They Work Together

The request lifecycle

Every feature follows the same three-tier path:

```
Browser (React)
   │  axios → services/api.js
   ▼
Node API (Express)
   route → validation middleware → controller
   controller:
     1. extract & validate GitHub username
     2. fetch + compress GitHub data  (services/*Analyzer)
     3. cache the expensive parts      (utils/cache.js)
     4. POST the compact payload → ML service
   ▼
ML Service (FastAPI)
   pydantic-validate → prompt_builder → llm_client (NVIDIA) → parse JSON
   ▼
   structured report flows back up the same path to the UI
```

Frontend modules

- **`Dashboard.jsx`** — the app shell and router. Listens to `popstate`, swaps pages with a `switch` over route constants, and passes a `navigate()` function down to pages.
- **`services/api.js`** — one Axios instance per backend feature (each with its own timeout, since AI calls can take minutes), plus one exported function per endpoint. A shared `wrapError()` normalizes timeouts, network failures, and server errors into a consistent `{ success, error }`.
- **`features/<name>/`** — each feature ships a custom hook (e.g. `useSpecializationFit`) that owns its state + API call, plus a `components/` folder for its form, skeleton loader, and report views.
- **`components/`** — shared, presentational pieces (`FeatureButton`, `BackButton`, `ErrorBoundary`, and the ATS-result cards).

Backend modules

- **`app.js`** — assembles the Express app: Helmet, compression, Morgan logging, CORS allow-list, JSON limits, rate limiter, route mounting, and a central error handler.

- **Controllers** — orchestrate one feature each. They validate the username, fan out GitHub work, call the ML service, and shape the response. They also translate upstream errors into clean HTTP codes (404 user-not-found, 429 rate-limit, 504 AI-timeout).
- **Services** — `github.service.js` (raw GitHub REST calls), `repoAnalyzer` / `githubDeepAnalyzer` (turn raw repos into a compact signal summary — READMEs, CI, Docker, tests, languages, commit counts), `scoring.service.js` (the 100-point score), and `shortlistAnalyzer` (bounded-concurrency batch analysis).
- **Utils** — `extractUsername` (URL → validated handle), `parseResume` (PDF/DOCX/TXT → text), `parseShortlistExcel` (sheet → rows), and `cache.js` (in-memory TTL cache that dedupes expensive GitHub + LLM work).

ML service modules

- **`main.py`** — declares every FastAPI endpoint and wires it to the matching feature package.
- **`llm_client.py`** — a lazily-initialized singleton OpenAI client pointed at NVIDIA, with streaming completion, one automatic retry, and helpers that strip code fences / `<think>` blocks and extract the JSON object from the model's reply.
- **Per-feature packages** — each has `schema.py` (Pydantic contracts), `prompt_builder.py` (system + user prompt), and `service.py` (calls the LLM and validates the parsed result back into the schema).

The scoring system (GitHub Profile Analysis)

Dimension	Weight	What it measures
Documentation Quality	25%	README presence across repos
Active Repositories	25%	Repos with meaningful history (>5 commits)
Language Diversity	20%	Variety of languages used
Community Impact	15%	Stars received
Commit Consistency	15%	Average commit depth per repo

8. Data Models

> **There is no database.** Gitfolio is stateless. The only persistence is a short-lived **in-memory TTL cache** in the Node process (`server/src/utils/cache.js`) that dedupes repeated GitHub/LLM calls. The "models" below are *data contracts* (Pydantic schemas), not DB tables.

In-memory cache

A capped `Map` (max 200 entries) of `key → { value, expires }`. GitHub deep-analyses are cached ~10 min per user; generated reports ~30 min per (user, role, job-description) tuple. Oldest entry is evicted when full. Resume text is **never** cached (per-upload and sensitive).

Key contract — GitHub Profile Summary

The compact profile the Node layer sends to the LLM (Pydantic):

```
GithubProfileSummary:
  username: str
  name, bio: str | null
  public_repos, followers, following: int
  account_age_years: float
  total_stars: int
  languages_distribution: { language: count }
  contribution_signal: "low" | "moderate" | "high"
```

Key contract — GitHub Repo Summary

Per-repo structured signals (the LLM never sees raw README bodies — only a truncated excerpt — for token cost and prompt-injection safety):

```
GithubRepoSummary:
  name, description, primary_language, license: str | null
  stars, forks, size_kb, commit_count: int
  languages, topics, detected_frameworks: [str]
  has_readme, has_ci, has_docker, has_kubernetes,
  has_tests, has_deployment, is_fork, is_archived: bool
  readme_excerpt, last_pushed: str | null
```

Key contract — Specialization Fit Report (output)

```
FitReport:
  role_id, role_name: str
  fit_score, confidence: int (0–100)
  score_breakdown: { dimension: int }
  strengths, weak_areas, missing_skills,
  matched_skills, recommended_improvements: [str]
  recommended_projects: [{ title, why, difficulty,
                           estimated_weeks, key_tech }]
  learning_path: [{ step, duration, resources }]
  hiring_readiness: { level, estimated_time_to_ready, rationale }
  radar: [{ axis, score }]
  interview_questions, ats_suggestions: [str]
  summary: str
```

The other features (`resume_match`, `github_compare`, `shortlist`) follow the same pattern: a Pydantic request schema in, a strongly-typed report schema out, validated on both sides.

9. Usage Guide

Once all three services are running, open **http://localhost:5173**. The home screen splits into two paths.

Student tools

1. **GitHub Profile Analysis** — paste a GitHub profile URL (or just a username) and submit. You'll get a 0–100 score with an animated ring, top repositories, strengths, red flags, and a personalized improvement roadmap.
2. **Specialization Fit Checker** — pick a target role, paste a job description, and enter your GitHub URL. You'll get a fit score, a radar chart of strengths/gaps, missing skills, recommended projects, and a learning path with a time-to-job-ready estimate.

Recruiter tools

1. **Candidate Shortlist** — upload an Excel file of candidates (with their GitHub URLs), paste the job description, choose how many to shortlist, and submit. The app analyzes each profile and returns an AI-ranked top N, plus a list of any profiles it had to skip.
2. **Resume vs GitHub Match** — upload a candidate's resume (PDF/DOCX/TXT) and their GitHub URL. The report shows whether the resume's claims are backed by real

repositories and an alignment score.

3. **GitHub Profile Compare** — enter two GitHub handles to get a side-by-side scored comparison.

> **Expect AI features to take time.** Single-profile analysis is < 60s; resume match and compare can take 1–2 minutes; a large shortlist can run several minutes (the client timeouts are set accordingly).

10. Common Issues & Troubleshooting

Symptom	Likely cause & fix
AI feedback is empty / "AI service is not configured"	The ML service isn't running or `ML_SERVICE_URL` is unset in `server/.env`. Start `uvicorn main:app --port 8000` and confirm the URL.
ML service errors with "NVIDIA_API_KEY is not set"	Add `NVIDIA_API_KEY` to `ml-service/.env` and restart uvicorn. The client is lazy-initialized, so the error only appears on the first AI call.
"GitHub API rate limit reached" (HTTP 429)	You're using unauthenticated GitHub access (60 req/hr). Add a `GITHUB_TOKEN` to `server/.env` to raise the limit to 5000 req/hr.
"GitHub user not found" (404)	The handle doesn't exist or the URL is malformed. The validator accepts a full URL or a bare username only.
CORS error in the browser console	The frontend origin isn't in `FRONTEND_URL` (server) or `CORS_ORIGINS` (ml-service). Add `http://localhost:5173` to both.
"Request timed out"	Large shortlists and cold caches are slow. Retry — the GitHub analysis is cached for ~10 min, so the second attempt is much faster.
Frontend can't reach the API	Check `VITE_API_URL` in `client/.env` and that the server is on port 5000. Restart Vite after changing any `VITE_` variable.
"Too many requests, please try again later."	You hit the 100-requests / 15-minute rate limit. Wait, or raise the limit in `server/src/app.js` for local testing.
Resume/Excel upload	Check the file type (resume: PDF/DOCX/TXT/MD; roster:

rejected	XLSX/XLS/XLSM) and that it's under 5 MB.
The LLM returns malformed JSON	The service already strips code fences and `<think>` blocks and retries once. Persistent failures usually mean an invalid model name in `NVIDIA_MODEL` or an expired key.
